

Improving Crackling Cloud's ISSL Functionality and Scalability

Final Research Report submitted for EGH400-2 Semester 2 2025

Student Name: [Zach, BARLOW]
Supervisor: [Jake, BRADFORD]
Submission Date: [20/10/25]

Abstract

Crackling Cloud is a serverless cloud application designed to identify CRISPR-Cas9 target sites within genomes using the Inverted Signature Slice Lists (ISSL) specialist data structure. While the application successfully preserved the accuracy of its local counterpart, its scalability was severely constrained by the AWS Lambda environment, limiting genome sizes to approximately 600 MB due to time and memory restrictions. This project investigated optimisations to Crackling Cloud's ISSL implementation to improve performance and scalability without compromising cost-effectiveness. Initial testing identified the `isslCreationLambda` function as the primary bottleneck, as it handled both off-target extraction and ISSL index creation within one Lambda invocation, resulting in excessive runtimes and frequent failures. To address this, the functionality was restructured into multiple Lambda functions, with the off-target extraction stage parallelised and reimplemented to reduce runtime and memory overhead. Additionally, significant memory optimisations were introduced into the ISSL index creation code, enabling buffered data processing rather than full in-memory storage. The final design increased the maximum genome size supported by the index-building phase to ~2.7 GB and improved run-time performance, while maintaining accuracy and the serverless cost model. However, a new bottleneck was identified in the ISSL scoring phase, restricting full pipeline execution to genomes of ~1.7 GB depending on sequence redundancy. The final design solution for Crackling Cloud's ISSL index building phase can be seen in the following public GitHub repository: <https://github.com/zbarl3/Crackling-AWS/tree/ZachBarlow-Capstone>

Table of Contents

Introduction	4
Literature Review	4
Cloud-based CRISPR-Cas9 Software Tools	4
AWS Services and their Expenses	4
Crackling Cloud and its ISSL Implementation	5
Identified Research Gap	5
Methodology	5
Initial Testing and Data Collection	5
Design	7
Management of Risks	10
Consideration of Ethics and Sustainability	11
Stakeholder Input	11
Results	12
Discussion	15
Conclusion	15
Sign off	16
References	16
Appendix A: Research Project Timeline	18
Appendix B: AWS CloudWatch Logs of Crackling Cloud Lambda Functions	19
Appendix C: Diagram of Previous Iteration of Crackling Cloud's Architecture	22
Appendix D: Diagram of Project's Solution within Crackling Cloud's Architecture	23

Introduction

Crackling Cloud is a cloud-based software tool that enables users to easily identify target gene sequences within genomes for use with CRISPR-Cas9 gene editing technology (Bradford, et al., 2024). The cloud application, based on the standalone Crackling tool (Bradford, et al., 2022), processes a user's requested guide RNA (gRNA) and identifies potential genomic sites where the Cas9 enzyme may bind. Since genomes can have up to millions and billions of sequences, there can be many off-target sites that share the same genomic sequence with the desired on-target site which can lead to unintended edits. For off-target identification and scoring, Crackling Cloud utilises a specialist data structure called "Inverted Signature Slice Lists" (ISSL) that is specifically designed for faster off-target scoring compared to other currently available CRISPR-Cas9 software tools. After thorough research and testing, the previous version of Crackling Cloud's implementation of ISSL index creation was found to be heavily restricted compared to its standalone counterpart due to the application's serverless architecture and the limitations that came with it. Thus, the aim of this project was to investigate and make optimisations to Crackling Cloud's ISSL implementation that enhanced its functionality, specifically focusing on ways to improve the scalability of its cloud adaptation whilst keeping costs to a minimum. To achieve this, the following objectives were clear:

- Determine any major bottlenecks within Crackling Cloud's serverless pipeline.
- Implement effective optimisations that improve the overall application's functionality whilst matching the previous Crackling Cloud's off-target score accuracy.
- Maintain the cost effectiveness of the original Crackling Cloud's AWS implementation.

Literature Review

Cloud-based CRISPR-Cas9 Software Tools

Numerous CRISPR-Cas9 software tools have been adapted into web or cloud applications to improve accessibility and usability. Tools such as CRISPAItRations (Kurgan, et al., 2021) and CHOPCHOP (Montague, et al., 2014) emphasise intuitive interfaces and visual summaries, while CRISPOR (Concordet & Haeussler, 2018) provides flexible data export options. Although online tools are practical for general users, studies indicate that bioinformaticians often favour offline tools due to superior performance and reduced runtime (Tycko, Myer, & Hsu, 2016). This is largely because cloud implementations must balance user accessibility with resource costs, limiting scalability. For example, CHOPCHOP is less suited to genomes with high redundancy (Mohammed Ajmal, et al., 2025), while E-CRISP recommends high-performance clusters for large-scale designs, where processing can take 12–16 hours (Heigwer & Boutros, 2020).

AWS Services and their Expenses

Crackling Cloud relies on AWS services, which follow a pay-as-you-go pricing model. Expenses typically arise from compute, storage, data transfer, and networking (Tatineni, 2019). Compute costs, often from EC2 instances, are particularly significant due to resource overprovisioning and inefficient scaling (Thota, 2023). Cost reduction strategies include auto-scaling policies and monitoring with AWS CloudWatch (Tatineni, 2019). Serverless solutions such as AWS Lambda have gained popularity for their ability to eliminate idle costs, with studies showing cost savings of up to 40% compared to EC2 instances (Thota, 2023). However, this benefit comes with performance trade-offs, such as cold start delays and memory/time limitations.

Crackling Cloud and its ISSL Implementation

Crackling Cloud’s architecture is serverless and event-driven, relying on AWS Lambda for genome processing (Bradford, et al., 2024). While accuracy matches the standalone tool, runtime performance lags significantly. For instance, constructing the *Oryza sativa* genome required only 3–14 seconds with the standalone tool (Bradford, et al., 2022), but 420 seconds in Crackling Cloud (Bradford, et al., 2024) although this value does include time to download and process the genome which is excluded in the previous benchmark’s measurements. This disparity is likely due to Lambda’s limitations: a 15-minute execution cap and a 10 GB memory limit. To overcome this, Crackling Cloud partitions genomes over 50 MB and processes them in parallel Lambda functions, which reduces failure risk but still results in slower performance. Studies also indicate that serverless architectures often suffer from performance degradation, raising the possibility that EC2-based designs may outperform Lambda at higher costs (Thota, 2023). This highlights the fundamental trade-off between cost efficiency and runtime performance.

Identified Research Gap

Previous research demonstrates that while Crackling Cloud successfully preserves the accuracy of the standalone Crackling tool, its runtime performance remains a bottleneck due to AWS Lambda limitations. Current literature has not sufficiently explored cost-efficient optimisations of ISSL in a cloud setting, particularly strategies that improve scalability without undermining accuracy or incurring significant additional costs. This project addresses that gap by investigating optimisation techniques to enhance Crackling Cloud’s ISSL implementation while balancing the trade-offs between performance and cost.

Methodology

Initial Testing and Data Collection

To fulfil the first objective outlined in the introduction, thorough testing of the original Crackling Cloud’s implementation was required to identify problem areas within the cloud pipeline. To get accurate testing results, the Crackling Cloud application had to be deployed to AWS so the results found would accurately represent how the software tool would run in its intended cloud environment. Once deployed, several test jobs were performed with different genomes of varying sizes. The genomes selected for testing were the same ones used to showcase the run-times in both the Crackling (Bradford, et al., 2022) and Crackling Cloud (Bradford, et al., 2024) publications. To help pinpoint sections that struggled the most when performing a job, the output logs for each Lambda function were recorded from AWS CloudWatch to explicitly find where Crackling Cloud was performing the worst in terms of run-time and memory usage. These measurements specifically matter the most when using AWS Lambda as each invocation of a Lambda function can only be active for 15 minutes and has a maximum memory size of 10 gigabytes where exceeding either of these limits causes the function to terminate without finishing its task leading to the whole job being left incomplete. Since the main focus of this project was primarily on the ISSL implementation of Crackling Cloud, the following table only showcases the Lambda functions that correlate to this functionality whereas the full measurements for every Lambda function invoked during a job can be seen in the table in Appendix B. A diagram detailing the full architecture of the previous Crackling Cloud implementation which outlines the order in which these Lambda functions are invoked during a job can be seen in Appendix C.

	Oryza Sativa (360MB)	Panicum Hallii (517MB)	Xiphophorus Couchianus (664MB)	Parus Major (985MB) (Failed)
isslCreationLambda (Indexes the Genome)	Billed Duration: 303520ms Max Memory Used: 6784 MB	Billed Duration: 547013ms Max Memory Used: 9971 MB	Billed Duration: 525235ms Max Memory Used: 10240 MB	Billed Duration: 755171ms Max Memory Used: 10240 MB Error: No space left on device
targetScan (Scans a sequence for CRISPR sites)	Billed Duration: 210ms Max Memory Used: 80 MB	Billed Duration: 190ms Max Memory Used: 81 MB	Billed Duration: 166ms Max Memory Used: 81 MB	N/A
Issl (Assesses gRNA specificity using ISSL)	Instance 1 Billed Duration: 29894ms Max Memory Used: 5009 MB Instance 2: Billed Duration: 29782ms Max Memory Used: 5005 MB Instance 3: Billed Duration: 30056ms Max Memory Used: 5005 MB	Instance 1 Billed Duration: 36745ms Max Memory Used: 6809 MB Instance 2 Billed Duration: 37743ms Max Memory Used: 6810 MB	Instance 1 Billed Duration: 62144ms Max Memory Used: 8966 MB Instance 2 Billed Duration: 60419ms Max Memory Used: 8963 MB	N/A
Consensus (Assesses gRNA efficiency)	Billed Duration: 128ms Max Memory Used: 529 MB	Instance 1 Billed Duration: 141ms Max Memory Used: 529 MB Instance 2 Billed Duration: 128ms Max Memory Used: 529 MB	Instance 1 Billed Duration: 139ms Max Memory Used: 528 MB Instance 2 Billed Duration: 119ms Max Memory Used: 529 MB	N/A

Table 1: AWS CloudWatch logs for ISSL Lambda functions where different genomes are listed above, and the different Lambda functions are listed on the left. Billed duration indicates a Lambda function's run-time in milliseconds and max memory used indicates how much memory was used by a Lambda function where 10240 MB (10GB) is the memory limit for any given Lambda function.

When looking at the AWS CloudWatch logs, it's clear the isslCreationLambda function struggles the most by far compared to the other Lambda functions. The run-times recorded for this Lambda function were very large compared to all other measurements found in the other functions, with the Parus Major genome having a run-time of approximately 12 and half minutes before failing due to exceeding the 10GB memory limit. As can be seen within the table, this failure within

isslCreationLambda caused the other Lambda functions to never be invoked leading to the job being left incomplete. Even if the Parus Major genome did not exceed the memory limit during this function call, it is still highly likely that the run-time limit would have been exceeded anyway. These results give a clear indication that this Lambda function serves as a bottle neck for the entire Crackling Cloud pipeline, as genomes that are close to 1GB in size or greater cannot be computed without isslCreationLambda failing. Overall, these tests proved successful in identifying a major bottleneck within Crackling Cloud's pipeline and served as a catalyst for the project's primary focus of optimising the isslCreationLambda function.

The next action taken was to perform a code review of this Lambda function to help gain insight into why isslCreationLambda fails the way it does. For this task, the standalone Crackling tool was used as a reference to help identify what operations were being performed by this function. This led to the discovery that not only did this Lambda function serve as the ISSL index creation stage, but also the stage in the application where it extracts all off-targets from the genome. This stage of the application is vital for the ISSL index creation phase as it provides a text file (called an offtargets file) that includes all possible Cas9 editable sequences from a genome's FASTA file and then sorts them so all duplicate sequences are located next to each other. This then allows the index building phase to efficiently search through the offtargets file to find all editable sequences and how many times they occur throughout the genome. Both of these phases in the Crackling tool are very time and memory expensive as it requires searching through upwards of millions and billions of sequences that result in files that are several times larger than the original FASTA input file which in of itself can be several gigabytes in size. After understanding this, it became very clear to see why this Lambda function would fail at such a small input size compared to the original Crackling tool. Not only is the extract off-targets phase of isslCreationLambda entirely sequential and written in Python which already takes a significant amount of time to complete, having the Lambda function then try and build the ISSL index within the same invocation would easily overexert the Lambda function's remaining resources. With this functionality identified, the project then moved on to designing and implementing optimisations for isslCreationLambda in hopes of relieving this bottleneck and increasing the maximum allowable size of genomes.

Design

The initial conceptual design plan proposed consisted of making three different optimisations to isslCreationLambda and combining all three into Crackling Cloud's pipeline. These being:

1. Splitting the functionality of extracting the offtargets from the genome (handled by extractOfftargets.py) and the construction of the ISSL index (handled by isslCreateIndex.cpp) into two separate Lambda functions.
2. Having the utility of extractOfftargets.py be ported over to C++ to enable parallelisation of its functionality as its sequential implementation in Python was very time expensive.
3. Enable parallelisation for both of these separate Lambda functions so multiple invocations of the same function could work concurrently which would help in avoiding the time and memory limitations of AWS Lambda.

A diagram of this conceptual design plan can be seen below to better illustrate how these optimisations would change the cloud pipeline.

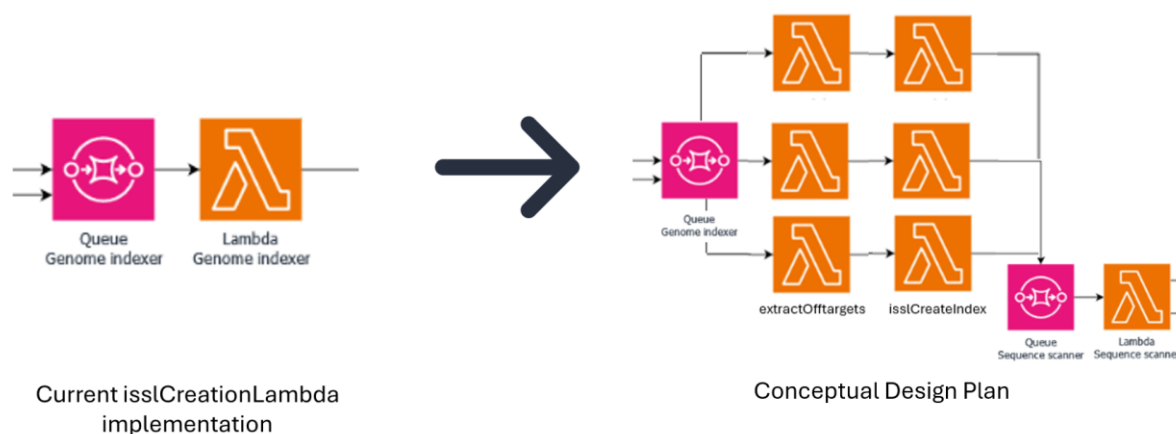


Figure 1: Current architecture implementation of isslCreationLambda within Crackling Cloud (left) and the resulting changes that would be made to isslCreationLambda's cloud architecture based on the conceptual design plan (right).

This design served as a solution to not only improve the run-time and memory usage of isslCreationLambda, but to also retain the entirely serverless nature of Crackling Cloud which would result in continuing to incur minimal costs compared to making use of other AWS services such as EC2 instances. This conceptual design plan hinged on the assumptions that spreading workloads across multiple concurrent Lambda invocations would still result in ISSL indexes that matched the layouts of how they were written in the original Crackling Cloud and that the off-target scoring would remain exactly the same as Crackling Cloud's previous implementation. When it came to implementing these optimisations, however, unanticipated issues arose that required the design plan to be reconsidered. For the first assumption, the ISSL index is encoded into binary by representing the nucleotides A, C, G, and T as the two-bit values 00, 01, 10, and 11 respectively. Because characters written in text take up 8 bits of file storage each, this decision to have the ISSL index written in binary allows the file to be a 4th of the size it would be if it was a simple text file. This then requires a unique way for the file to actually be read into memory, and this was achieved by having the ISSL index file write its data in segments and have a header section at the top that allows the scorer to effectively search through the binary file. This design of the ISSL index demands that all the data written to the file is written in the exact order outlined in isslCreateIndex.cpp otherwise it can cause the file to become corrupted and hence give invalid scores. This requirement led to the realisation that having isslCreateIndex be performed over multiple concurrent Lambda invocations would be near impossible to implement, as writing the ISSL index requires full access to all the data present in the offtargets file as well as coordination for writing the data in the correct locations which is extremely difficult to manage across multiple Lambda invocations. Along with this the design was initially tested with the extractOfftargets.cpp file found in Crackling++, a faster variant of the original Crackling tool that tries to take full advantage of the C++ language (Bradford, et al., 2025), where it was found that the off-target scores being produced differed heavily from the original Crackling Cloud implementation. These issues led to a new design plan that worked around these complications whilst still meeting the project's needs. The following diagram showcases the final design that was fully implemented and tested to successfully meet the project's requirements.

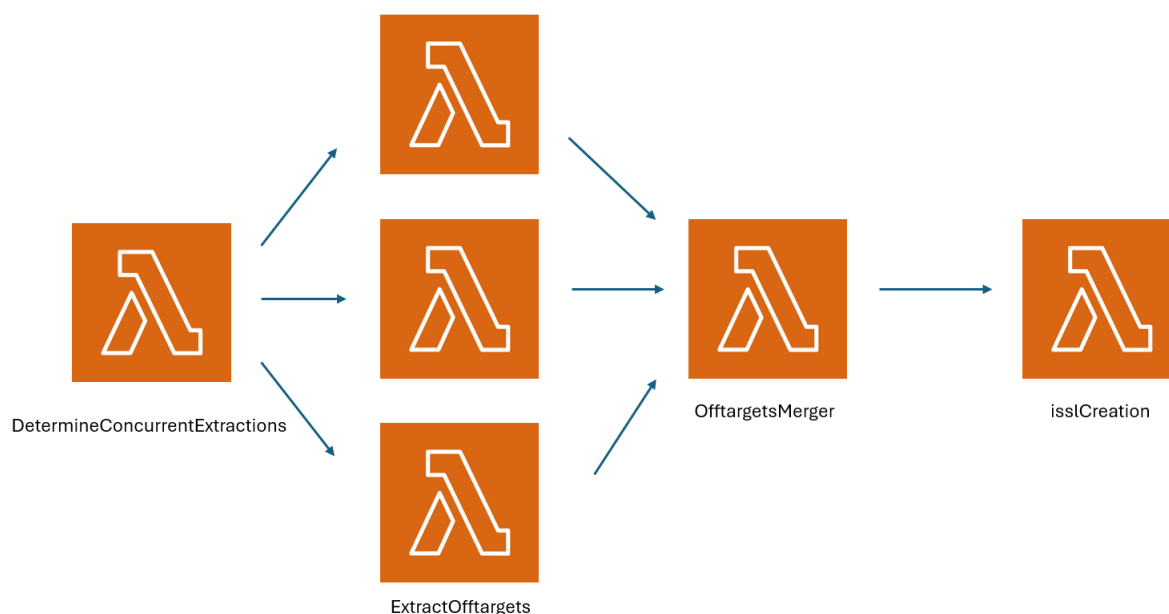


Figure 2: Final design for the optimisations implemented into isslCreationLambda's cloud architecture.

Here you can see several changes that have been made to isslCreationLambda's cloud architecture that differ from the conceptual design plan initially proposed. To begin with, the extractOfftargets.py functionality is now handled by three different Lambda functions. The first being DetermineConcurrentExtractions, which downloads the FASTA file of the genome and then determines how many headers are present within the file. It then sends ten SQS messages that invoke ten instances of the ExtractOfftargets Lambda function where the workload of the original extractOfftargets.py file is split between these ten Lambda functions concurrently instead of all in one Lambda invocation. To ensure the workloads between each ExtractOfftargets invocation is split evenly, each header is assigned to their corresponding invocation using round-robin scheduling, as many of the FASTA files being worked on usually have around several thousand headers present where the significant majority of the file size is occupied by only around the first 10 headers. In terms of the computation being performed by the ExtractOfftargets Lambda functions, it utilises the same functionality as extractOfftargets.py used in the original Crackling Cloud implementation as maintaining the scores of the original version was paramount. The only difference in the functionality is that the offtargets file is now written to binary instead of a text file as to make use of the file size reduction that was achieved with the ISSL index. Once these invocations finish extracting and sorting all their offtargets into lexicographical order, the resulting files are then uploaded to s3. Each ExtractOfftargets invocation updates a DynamoDB table that keeps track of how many invocations have completed their task, and then once the last invocation finishes and checks the DynamoDB table it then sends an SQS message to instantiate the OfftargetsMerger Lambda function. This function then simply downloads all the files that were uploaded to s3 by these ExtractOfftargets invocations and then merges them all into one offtargets binary file that maintains a lexicographical order.

For the ISSL index building phase, instead of finding a way to enable concurrency for this Lambda function, another method was devised and successfully implemented. When initial implementation and testing of the conceptual design was being performed for this Lambda function, some key characteristics were identified. Namely that although this function was executing its task on the previously tested genomes with a run-time of a mere minute which was a vast improvement from its predecessor, the max memory usage found within the AWS CloudWatch logs suggested severe

inefficient memory management within the `isslCreateIndex.cpp` file as the Parus Major genome was still failing at this stage of Crackling Cloud due to exceeding AWS Lambda's memory limits. After further investigation, it was found that within `isslCreateIndex.cpp` the program would do two actions that heavily increased memory usage. The first being that the inputted `offtargets` file was having its data read all at once and stored within a vector for the duration of the program, and then during slice computation a giant 3D vector was being reserved for all the lists of slices that would then be written to file at the end of the program. Both of these design choices served as a way to increase the total memory usage of the program astronomically, and within a Lambda function which already is housing these `offtargets` and `ISSL` files that are both several gigabytes in size would easily lead to the function instantly exceeding the AWS memory limits. To optimise this, the `isslCreateIndex.cpp` file was changed so that it only ever reads one sequence at a time from the `offtargets` file and then only reserves space for one slice list where it immediately writes the data to file after computation. The first optimisation was relatively easy to implement as it did not alter anything within the program, however for the second change, some of the program had to be altered in order to preserve the original order of the `ISSL` index. This involved moving the computation for the precalculated scores before the slice lists computation as the data for those scores is written to file before the slices are, and then the slice lists computation which originally took advantage of OpenMP to run its iterations in parallel had to be altered so it runs sequentially in order to avoid corrupting the `ISSL` index when writing each slice list iteratively.

Then finally when the `isslCreation` Lambda function is working within Crackling Cloud's pipeline, it will be invoked twice. For the first time, it is invoked by the `OfftargetsMerger` function where it will download the `offtargets` binary file, scan it for all the sequences and how many times each one occurs in the genome, and then writes that data to the `ISSL` file and prematurely uploads it to `s3` where it then sends an `SQS` message to reinvoke the function. Then during the second invocation, it downloads the `ISSL` file without redownloading the `offtargets` file and then continues to build the index from there. This design choice allows for the Lambda function to never hold the full file sizes of both the `offtargets` and `ISSL` files at the same time which helps to further avoid the maximum memory limit. Overall, these optimisations to both `extractOfftargets` and `isslCreateIndex` were successfully implemented into Crackling Cloud's serverless pipeline and a cloud diagram showcasing how these changes fit into the existing pipeline can be seen in Appendix D.

Management of Risks

During the designing and implementation of the project's methodology, no precautionary measurements were taken relating to health and safety risks. This is because the activities involved with this project only entailed programming and computing which did not arise any immediate or obvious concerns with regards to health and safety. One of the risks that was mentioned in previous documentation was the possibility of overcharging the provided AWS account as it was a very realistic risk that could occur if no healthy cloud computing habits were put into practice. Since the conceptual and final designs revolved around maintaining the entirely serverless and event-driven architecture of Crackling Cloud, potential issues with incurring idle costs were not considered as a cause for concern. Deployments and tests performed during the designing and implementation phases of the project were kept to a minimum to help avoid risks of overcharging the AWS account with needless jobs.

With consideration to the risk mentioned in previous documentation of potentially running out of time to finish the project, this possibility was at the forefront of decision making when designing

optimisations for Crackling Cloud. The original scope of the project was to focus solely on improving the `isslCreationLambda` function as it served as the immediate bottleneck that limited the rest of the application heavily. This decision allowed for much more time to be dedicated to producing an effective solution to this problem whilst not having to dedicate time to other issues noticed originally that were not immediately necessary to fix. This decision also gave room for the possibility of a need to rescope the conceptual design which did end up occurring during the latter half of the project's duration. Risks to code were also just as important as time, as maintaining the accuracy of the original Crackling Cloud was vital since providing incorrect on-target and off-target predictions can lead to adverse and undesirable affects if bioinformaticians were to use CRISPR-Cas9 gene editing technology with incorrect computed data. Thankfully, the final implementation of this project resulted in exactly the same scores as the previous iteration of Crackling Cloud, leading to signs that changes to the code base has resulted in no adverse or unintended behaviour and validating the final design as a whole.

Consideration of Ethics and Sustainability

The ethics of the decision-making process were a significant factor throughout the project. Prior to committing to any decision, efforts were made to ensure that choices were as well-informed as possible. Feedback and recommended strategies from the supervisor, Jake Bradford, played a key role in determining the most appropriate course of action for many tasks. Regular consultation during weekly meetings provided informed insights that guided decisions in accordance with Section 2: Practise Competently of the Engineers Australia Code of Ethics and Guidelines of Professional Conduct.

Section 1: Demonstrate Integrity was upheld during these weekly meetings. Respecting the opinions of others and maintaining honesty regarding project progression fostered trust and contributed to the advancement of the project.

Furthermore, Section 4: Promote Sustainability was applied through the integration of sustainable development principles into the decision-making process. Carbon emissions are a recognised concern in cloud computing, and Crackling Cloud was designed as a serverless, event-driven application which helped in minimising environmental impact. Maintaining this approach ensured that no emissions were produced when the tool was inactive, in contrast to alternatives such as EC2 instances, which consume resources even while idling and therefore contribute to unnecessary emissions.

Stakeholder Input

Stakeholder input was an important aspect of the project. Supervisor Jake Bradford provided consistent support during weekly meetings and was instrumental in guiding the project in the right direction. He supplied necessary resources and contributed valuable suggestions and recommendations that shaped the project's final plan. Throughout the duration of the project, direct user testing by bioinformaticians and other end-users of Crackling and Crackling Cloud had not been undertaken. However, such testing was identified as a beneficial step toward ensuring the success of the project and would be encouraged if further development of the project took place.

Results

The final design's implementation into Crackling Cloud's pipeline has resulted in several improvements to the application's performance and usability. To begin with, the most notable improvement was that the ISSL index building phase can now work with genomes of up to ~2.7GB in size compared to its previous cut off of 600MB, leading to a ~4.5x increase in maximum input file size. This is a breakthrough for the application as this severe input size limitation left the software tool in a state where its usability was heavily limited due to the fact that genomes more often than not exceeded the 600MB file size limit with ease. Now although the increase in input file size is substantial for the ISSL index building phase, the input file size for the Crackling Cloud application as whole was found to be smaller in comparison. When testing several more genomes from the original Crackling publication (Bradford, et al., 2022), a new bottleneck was discovered relating to the ISSL scoring phase. The following table outlines all the genomes that successfully made it through the index building phase and the resulting AWS CloudWatch logs for the Lambda functions relevant to the indexing and scoring stages.

D = duration in ms M = memory used in MB	Offtargets Scheduler	Offtargets Extractor (last invocation to complete job)	Offtargets Merger	ISSL Index Builder (Highest recorded invocation of the two phases)	ISSL Scorer (last invocation to complete job)
Oryza Sativa (360MB)	D = 3515.73 M = 597	D = 49624.02ms M = 1035	D = 70902.90 M = 747	D = 60520.63 M = 3296	D = 30184.14 M = 5006
Panicum Hallii (517MB)	D = 5374.28 M = 806	D = 88008.4 M = 1557	D = 112382.63 M = 1095	D = 75165.49 M = 3806	D = 38130.50 M = 6810
Xiphophorus Couchianus (664MB)	D = 6539.00 M = 1012	D = 62698.64 M = 1347	D = 113269.45 M = 1138	D = 121926.70 M = 6332	D = 63457.36 M = 8972
Parus Major (985MB)	D = 9139.78 M = 1447	D = 435508.84 M = 4054	D = 204591.70 M = 1892	D = 238547.57 M = 10227	D = 129867.69 M = 10228
Crotalus Viridis (1297MB)	D = 11261.85 M = 1854	D = 588471.72 M = 6637	D = 196998.33 M = 2129	D = 225696.48 M = 10228	D = 123759.79 M = 10229
Danio Rerio (1679MB)	D = 14999.49 M = 2313	D = 294819.61 M = 3247	D = 254212.77 M = 2259	D = 176011.57 M = 8667	D = 121025.81 M = 10230
Oncorhynchus Mykiss (1950MB)	D = 18305.97 M = 2897	D = 536638.47 M = 4364	D = 387574.87 M = 3353	D = 272069.97 M = 10228	Failed due to exceeding memory limit
Mus Musculus (2730MB)	D = 25122.51 M = 3669	D = 777827.34 M = 6964	D = 496332.81 M = 4926	D = 522032.41 M = 10228	Failed due to exceeding memory limit

Table 2: AWS CloudWatch logs for optimised ISSL Lambda functions where different genomes are listed on the left, and the different Lambda functions are listed above. Duration indicates a Lambda function's run-time in milliseconds and memory used indicates how much memory was used by a Lambda function.

After performing these jobs, it can be seen that although the ISSL index building phase was able to produce an ISSL index for the *Oncorhynchus Mykiss* and *Mus Musculus* genomes, the ISSL scorer Lambda functions failed when processing these genomes due to exceeding the Lambda memory limit. This leaves the maximum input file size for the application as a whole at roughly 1.7GB as the *Danio Rerio* genome was the largest file to make it through the ISSL scorer. This discovery of a new bottleneck for the application is likely due to the file sizes of the ISSL indexes for these genomes being too big for the ISSL scorer to handle. The following table details the output file sizes for both the offtargets files and the ISSL indexes for all tested genomes along with the counts for their total sequences and total unique sequences found.

	Offtargets file size	ISSL index file size	Total sequences	Total unique Sequences
Oryza Sativa (360MB)	293.8MB	2GB	61606361	45737176
Panicum Hallii (517MB)	462.3MB	2.4GB	96958708	53916313
Xiphophorus Couchianus (664MB)	481.9MB	4GB	101066369	89884314
Parus Major (985MB)	844.4MB	7.7GB	177084534	171932270
Crotalus Viridis (1297MB)	959.3MB	7.2GB	201173374	160657249
Danio Rerio (1679MB)	1GB	5.5GB	214767384	124089653
Oncorhynchus Mykiss (1950MB)	1.5GB	8.5GB	326107795	189235322
Mus Musculus (2730MB)	2.3GB	10GB	486163861	370940062

Table 3: ISSL index and offtargets file sizes for all tested genomes along with the counts for their total sequences and total unique sequences found.

Within this data, a noticeable trend can be seen with the ISSL index file sizes being proportional to how many unique sequences are present within a genome. This trend indicates that the Crackling Cloud application could likely handle genomes bigger than 1.7GB, however, these genomes would need to have a high rate of redundancy in the sequences present to produce small enough ISSL

indexes for the scorer to handle. Either way, table 2 and table 3 provide insight into why the ISSL scoring phase of the software tool fails which is that it is majorly dependant on how big the ISSL index file is.

As for the performance enhancements achieved, the ISSL index building phase proves to be faster than the previous iteration's `isslCreationLambda` function. The following table details the run-times achieved in both ISSL index building phases in the original Crackling Cloud and the project's solution. The times recorded for the project's design include the Lambda functions `DetermineConcurrentExtractions`, the last invocation of `ExtractOfftargets` to complete its task, `OfftargetsMerger`, and both invocations of `isslCreation` as the combination of these Lambda functions mimic the functionality performed in `isslCreationLambda` from the previous iteration.

Run-time in seconds	isslCreationLambda (Previous iteration)	Project's solution
Oryza Sativa (360MB)	303s	224s
Panicum Hallii (517MB)	547s	329s
Xiphophorus Couchianus (664MB)	525s	386s
Parus Major (985MB)	N/A	1043s
Crotalus Viridis (1297MB)	N/A	1166s
Danio Rerio (1679MB)	N/A	850s
Oncorhynchus Mykiss (1950MB)	N/A	1385s
Mus Musculus (2730MB)	N/A	2055s

Table 4: Run-times measured for both ISSL index building phases for the previous iteration of Crackling Cloud and the Project's Solution in seconds.

Comparing the genomes that were able to be completed by the previous iteration's `isslCreationLambda`, the final design for the ISSL index building phase was able to achieve an average speed up of 1.45x over the original Crackling Cloud's ISSL index building phase. After achieving these results, the supervisor Jake Bradford was disclosed on the improvements made and was satisfied with the project's achievements.

Discussion

Considering the scope of the project, the solution provided can be considered a relative success as it achieved its goal of improving Crackling Cloud's ISSL functionality. The optimisations made enhanced its scalability by enabling concurrency while also improving its memory efficiency, leading to an ISSL implementation that is better suited to the AWS cloud infrastructure than the previous iteration. Maintaining the original Crackling Cloud's entirely serverless and event driven architecture also achieved its goal of keeping costs to a minimum as the compute costs for the final design would be relatively the same as the original's. Making use of other services like AWS EC2 definitely had the potential to improve Crackling Cloud's capabilities beyond what was achieved here, however, these services can easily increase the overall costs of the application so the decision to maintain the entirely serverless pipeline feels justified. To then finally validate these changes, the scores calculated for the genomes that were able to be processed by the original Crackling Cloud were matched exactly by the final design's implementation.

The glaring issue with this solution is the bottleneck found within the ISSL scorer Lambda function. Although the final design's implementation was able to work with a genome with a file size of 2.7GB, the ISSL scorer could only get through a genome with a file size of 1.7GB with anything above that resulting in failure. It was found that this function was mainly failing due to how big the ISSL index files became, as ISSL file sizes of around 8GB only left the scorer to work with the remaining 2GB of memory within the Lambda instance which in its current iteration it could not work with. There are two potential ways the application could be optimised in order to relieve its bottleneck, these being to either change the code within the ISSL scorer function to be more memory efficient or find a way to reduce the overall file size of the ISSL data structure. The first optimisation would be the easiest to implement, however, implementing this change would likely lead to minimal improvement as there is such little amount of memory to work with between the ISSL index and the rest of the Lambda invocation's memory space. The second optimisation, on the other hand, doesn't seem relatively feasible as the ISSL data structure is already written in binary which allows for the smallest amount of file storage possible for each piece of data. Either way, there is definitely still room to explore exactly how this bottleneck could be removed, however, the amount of time it would take to determine and implement these changes just makes it impossible to have inside the scope of the project.

The outcome of the project would not have been possible if the considerations to risk, ethics, and sustainability weren't managed as well as they were. The potential risks that were identified were handled exactly as they were described in the Methodology section, which proved effective in ensuring the success of the final design. The ethics and sustainability measures also outlined in the Methodology section were upheld throughout the entirety of the project's duration which further contributed to the project's success.

Conclusion

In summary, the final design's solution to the project's problem statement was a relative success. The optimisations made to Crackling Cloud's ISSL implementation improved upon the software tool's overall run-time, usability and scalability whilst keeping costs to a minimum by maintaining its entirely serverless architecture. If the project was to be continued further, exploration into the

possible optimisations that could be made to the ISSL scorer Lambda function to relieve the newly discovered bottleneck would be the next recommended course of action.

Sign off

	SIGNATURE	DATE
STUDENT ENGINEER	Zach Barlow	20/10/2025

References

- Bradford, J., Chappell, T., & Perrin, D. (2022, June 20). Rapid Whole-Genome Identification of High Quality CRISPR Guide RNAs with the Crackling Method. *The CRISPR Journal*, 410-421. Retrieved April 13, 2025, from <https://www.liebertpub.com/doi/abs/10.1089/crispr.2021.0102>
- Bradford, J., Joy, D., Winsen, M., Meurant, N., Wilkins, M., Wilson, L., . . . Perrin, D. (2024, December 5). *Crackling Cloud: an event-driven, cloud-based CRISPR-Cas9 guide RNA design tool*. Retrieved April 2, 2025, from bioRxiv: <https://www.biorxiv.org/content/10.1101/2024.12.04.626718v1.article-info>
- Bradford, J., Schmitz, C., Salomone, R., & Perrin, D. (2025, January 10). *Fast and scalable off-target assessment for CRISPR guide RNAs using partial matches*. Retrieved from IEEE Xplore: <https://ieeexplore.ieee.org/abstract/document/10822812>
- Concordet, J.-P., & Haeussler, M. (2018, May 14). *CRISPOR: intuitive guide selection for CRISPR/Cas9 genome editing experiments and screens*. Retrieved April 2, 2025, from Oxford Academic: <https://academic.oup.com/nar/article/46/W1/W242/4995687?login=false>
- Heigwer, F., & Boutros, M. (2020, September 15). *Cloud-Based Design of Short Guide RNA (sgRNA) Libraries for CRISPR Experiments*. Retrieved April 2, 2025, from Springer Nature Link: https://link.springer.com/protocol/10.1007/978-1-0716-0687-2_1
- Kurgan, G., Turk, R., Li, H., Behlke, M. A., Wang, Y., McNeill, M. S., . . . Florus, K. (2021, June 11). *CRISPAItRations: A validated cloud-based approach for interrogation of double-strand break repair mediated by CRISPR genome editing*. Retrieved April 2, 2025, from Molecular Therapy: [https://www.cell.com/molecular-therapy-family/methods/fulltext/S2329-0501\(21\)00064-4](https://www.cell.com/molecular-therapy-family/methods/fulltext/S2329-0501(21)00064-4)
- Mohammed Ajmal, C. P., Keerthana, S., Hemalatha, N., & Ameer Suhail, K. (2025, January 24). *Development of CRISPR-Cas9 Algorithm for Designing gRNAs for Polyploid Sugarcane*. Retrieved April 2, 2025, from Springer Nature Link: <https://link.springer.com/article/10.1007/s12355-024-01523-9>
- Montague, T. G., Cruz, J. M., Gagnon, J. A., Church, G. M., & Valen, E. (2014, May 26). *CHOPCHOP: a CRISPR/Cas9 and TALEN web tool for genome editing*. Retrieved April 2, 2025, from Oxford Academic: <https://academic.oup.com/nar/article/42/W1/W401/2437392>
- Tatineni, S. (2019, December). *COST OPTIMIZATION STRATEGIES FOR NAVIGATING THE ECONOMICS OF AWS CLOUD SERVICES*. Retrieved April 13, 2025, from ResearchGate:

https://www.researchgate.net/publication/377402103_COST_OPTIMIZATION_STRATEGIES_FOR_NAVIGATING_THE_ECONOMICS_OF_AWS_CLOUD_SERVICES

Thota, R. C. (2023, November). *Cost optimization strategies for micro services in AWS: Managing resource consumption and scaling efficiently*. Retrieved April 13, 2025, from ResearchGate: https://www.researchgate.net/publication/389905288_Cost_optimization_strategies_for_micro_services_in_AWS_Managing_resource_consumption_and_scaling_efficiently

Tycko, J., Myer, V. E., & Hsu, P. D. (2016, August 4). *Methods for Optimizing CRISPR-Cas9 Genome Editing Specificity*. Retrieved April 2, 2025, from Molecular Cell: [https://www.cell.com/molecular-cell/fulltext/S1097-2765\(16\)30331-8](https://www.cell.com/molecular-cell/fulltext/S1097-2765(16)30331-8)

Appendix A: Research Project Timeline

Number	Focus	Deliverable	Dependency	Release Date/ Milestone
1	Literature Review	Literature Review Report		16/04/2025
2	Regular status updates	Regular status updates reported to supervisor and CRISPR-Cas9 peers		Weekly basis
3	Code Reviews and deployment	Results from code reviews for Crackling and Crackling Cloud and deploying current iteration		9/05/2025
4	Conceptual Design Plan	Plan for implementation changes to Crackling Cloud	1, 3	21/05/2025
5	Progress Report and Oral Presentation	A written report and oral presentation detailing the project's progress	4	28/05/2025
6	Beginning of implementing optimisations	Begin implementing optimisations to Crackling Cloud following conceptual plan	4	21/07/2025
7	Rescoping of design plan	Shifted conceptual design plan into the final design proposed	6	8/9/2025
8	Completion of final design	After experimentation and testing, final design is completed and ready for implementation	7	29/09/2025
9	Supervisor Consultation	Affirmation from supervisor that updated Crackling Cloud is approved to be uploaded into GitHub repository	2, 8	3/10/2025
10	Project Delivery Report & Oral Defence	A written report and oral defence detailing the project's outcomes	8, 9	22/10/2025

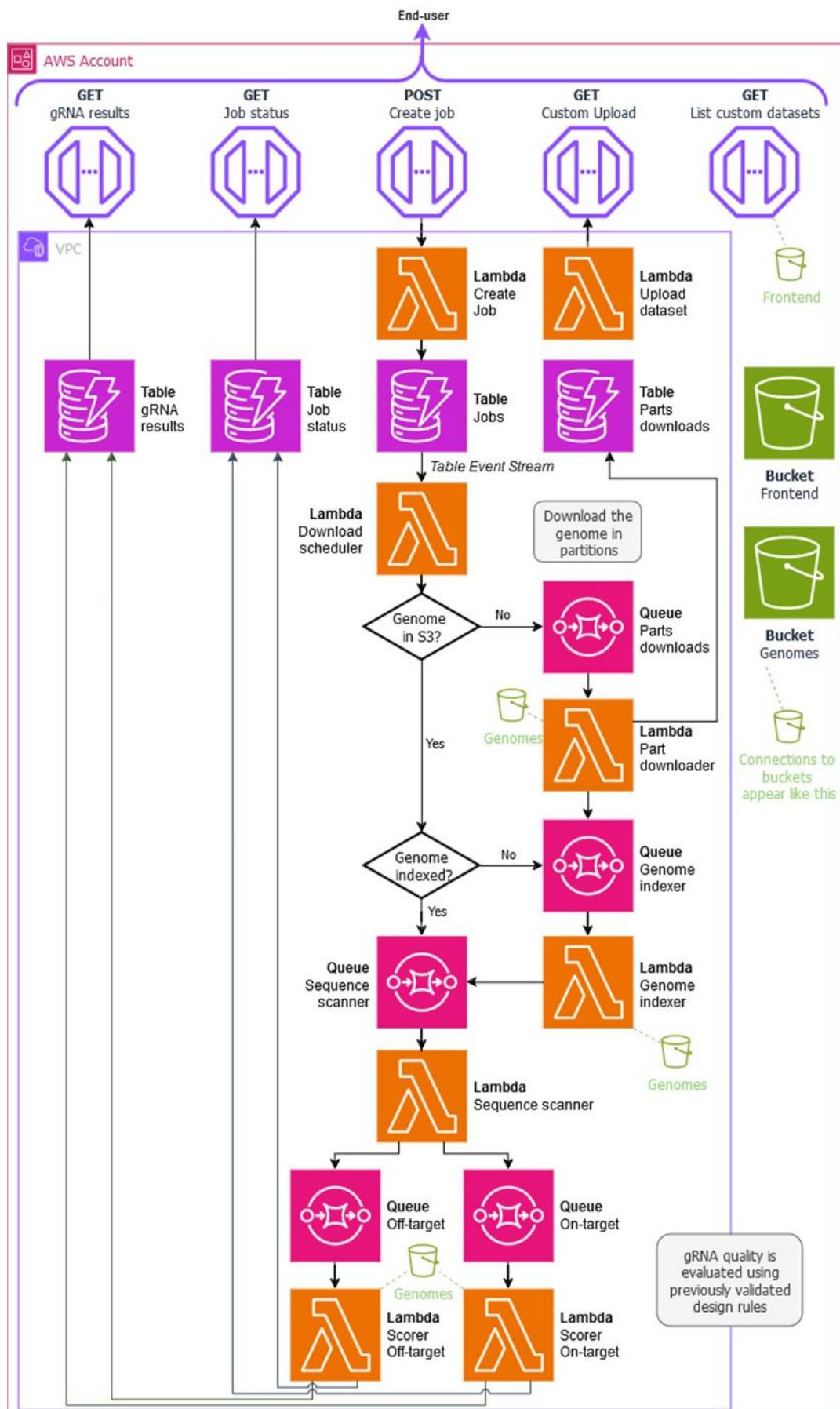
Appendix B: AWS CloudWatch Logs of Crackling Cloud Lambda Functions

	Oryza Sativa (360MB)	Panicum Hallii (517MB)	Xiphophorus Couchianus (664MB)	Parus Major (985MB) (Failed)
downloader	Total Threads: 1 Thread 1 Billed Duration: 169ms Memory Size: 2065 MB Max Memory Used: 89 MB Init Duration: 700.74ms	Total Threads: 1 Thread 1 Billed Duration: 152ms Memory Size: 2065 MB Max Memory Used: 89 MB Init Duration: 1146.36ms	Total Threads: 1 Thread 1 Billed Duration: 156ms Memory Size: 2065 MB Max Memory Used: 89 MB Init Duration: 594.82ms	Total Threads: 1 Thread 1 Billed Duration: 169ms Memory Size: 2065 MB Max Memory Used: 89 MB Init Duration: 684.85ms
issl	Total Threads: 3 Thread 1 Billed Duration: 29894ms Memory Size: 10240 MB Max Memory Used: 5009 MB Init Duration: 509.88ms Thread 2 Billed Duration: 29782ms Memory Size: 10240 MB Max Memory Used: 5005 MB Init Duration: 816.67ms Thread 3 Billed Duration: 30056ms Memory Size: 10240 MB Max Memory Used: 5005 MB	Total Threads: 2 Thread 1 Billed Duration: 36745ms Memory Size: 10240 MB Max Memory Used: 6809 MB Init Duration: 440.47ms Thread 2 Billed Duration: 37743ms Memory Size: 10240 MB Max Memory Used: 6810 MB Init Duration: 550.63ms	Total Threads: 2 Thread 1 Billed Duration: 62144ms Memory Size: 10240 MB Max Memory Used: 8966 MB Init Duration: 571.33ms Thread 2 Billed Duration: 60419ms Memory Size: 10240 MB Max Memory Used: 8963 MB Init Duration: 1016.38ms	N/A

	Init Duration: 603.58ms			
partloader	N/A	N/A	N/A	N/A
targetScan	Total Threads: 1 Thread 1 Billed Duration: 210ms Memory Size: 10240 MB Max Memory Used: 80 MB Init Duration: 366.83ms	Total Threads: 1 Thread 1 Billed Duration: 190ms Memory Size: 10240 MB Max Memory Used: 81 MB Init Duration: 741.47ms	Total Threads: 1 Thread 1 Billed Duration: 166ms Memory Size: 10240 MB Max Memory Used: 81 MB Init Duration: 355.88ms	N/A
isslCreationLambda	Total Threads: 1 Thread 1 Billed Duration: 303520ms Memory Size: 10240 MB Max Memory Used: 6784 MB Init Duration: 1042.79ms	Total Threads: 1 Thread 1 Billed Duration: 547013ms Memory Size: 10240 MB Max Memory Used: 9971 MB Init Duration: 407.53ms	Total Threads: 1 Thread 1 Billed Duration: 525235ms Memory Size: 10240 MB Max Memory Used: 10240 MB Init Duration: 423.95ms	Total Threads: 1 Thread 1 (Failed) Billed Duration: 755171ms Memory Size: 10240 MB Max Memory Used: 10240 MB Init Duration: 549.80ms Error: No space left on device
createJob	Total Threads: 1 Thread 1 Billed Duration: 251ms Memory Size: 128 MB Max Memory Used: 79 MB Init Duration: 440.24ms	Total Threads: 1 Thread 1 Billed Duration: 236ms Memory Size: 128 MB Max Memory Used: 79 MB Init Duration: 429.02ms	Total Threads: 1 Thread 1 Billed Duration: 257ms Memory Size: 128 MB Max Memory Used: 79 MB Init Duration: 436.43ms	Total Threads: 1 Thread 1 Billed Duration: 271ms Memory Size: 128 MB Max Memory Used: 79 MB Init Duration: 467.44ms
consensus	Total Threads: 1 Thread 1 Billed Duration:	Total Threads: 2 Thread 1 Billed Duration:	Total Threads: 2 Thread 1 Billed Duration:	N/A

	128ms Memory Size: 10240 MB Max Memory Used: 529 MB Init Duration: 7485.94ms	141ms Memory Size: 10240 MB Max Memory Used: 529 MB Init Duration: 9027.00ms Thread 2 Billed Duration: 128ms Memory Size: 10240 MB Max Memory Used: 529 MB Init Duration: 9004.23ms	139ms Memory Size: 10240 MB Max Memory Used: 528 MB Init Duration: 7945.06ms Thread 2 Billed Duration: 119ms Memory Size: 10240 MB Max Memory Used: 529 MB Init Duration: 7970.16ms	
--	--	--	--	--

Appendix C: Diagram of Previous Iteration of Crackling Cloud's Architecture



Appendix D: Diagram of Project's Solution within Crackling Cloud's Architecture

